JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.zip

Class Inflater

java.lang.Object
    java.util.zip.Inflater

```
public class Inflater
extends Object
```

This class provides support for general purpose decompression using the popular ZLIB compression library. The ZLIB compression library was initially developed as part of the PNG graphics standard and is not protected by patents. It is fully described in the specifications at the java.util.zip package description.

The following code fragment demonstrates a trivial compression and decompression of a string using `Deflater` and `Inflater`.

```
try {
    // Encode a String into bytes
    String inputString = "blahblahblah??";
    byte[] input = inputString.getBytes("UTF-8");

    // Compress the bytes
    byte[] output = new byte[100];
    Deflater compresser = new Deflater();
    compresser.setInput(input);
    compresser.finish();
    int compressedDataLength = compresser.deflate(output);

    // Decompress the bytes
    Inflater decompresser = new Inflater();
    decompresser.setInput(output, 0, compressedDataLength);
    byte[] result = new byte[100];
    int resultLength = decompresser.inflate(result);
    decompresser.end();
```

```
        // Decode the bytes into a String
        String outputString = new String(result, 0, resultLength, "UTF-8");
    } catch(java.io.UnsupportedEncodingException ex) {
        // handle
    } catch (java.util.zip.DataFormatException ex) {
        // handle
    }
```

See Also:

Deflater

Constructor Summary

Constructors

Constructor	Description
Inflater()	Creates a new decompressor.
Inflater(boolean nowrap)	Creates a new decompressor.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
void	**end**()	Closes the decompressor and discards any unprocessed input.
protected void	**finalize**()	Closes the decompressor when garbage is collected.
boolean	**finished**()	Returns true if the end of the compressed data stream has been reached.
int	**getAdler**()	Returns the ADLER-32 value of the uncompressed data.
long	**getBytesRead**()	Returns the total number of compressed bytes input so far.

long	**getBytesWritten**()	Returns the total number of uncompressed bytes output so far.
int	**getRemaining**()	Returns the total number of bytes remaining in the input buffer.
int	**getTotalIn**()	Returns the total number of compressed bytes input so far.
int	**getTotalOut**()	Returns the total number of uncompressed bytes output so far.
int	**inflate**(byte[] b)	Uncompresses bytes into specified buffer.
int	**inflate**(byte[] b, int off, int len)	Uncompresses bytes into specified buffer.
boolean	**needsDictionary**()	Returns true if a preset dictionary is needed for decompression.
boolean	**needsInput**()	Returns true if no data remains in the input buffer.
void	**reset**()	Resets inflater so that a new set of input data can be processed.
void	**setDictionary**(byte[] b)	Sets the preset dictionary to the given array of bytes.
void	**setDictionary**(byte[] b, int off, int len)	Sets the preset dictionary to the given array of bytes.
void	**setInput**(byte[] b)	Sets input data for decompression.
void	**setInput**(byte[] b, int off, int len)	Sets input data for decompression.

Methods inherited from class java.lang.Object

clone, equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

Inflater

```
public Inflater(boolean nowrap)
```

Creates a new decompressor. If the parameter 'nowrap' is true then the ZLIB header and checksum fields will not be used. This provides compatibility with the compression format used by both GZIP and PKZIP.

Note: When using the 'nowrap' option it is also necessary to provide an extra "dummy" byte as input. This is required by the ZLIB native library in order to support certain optimizations.

Parameters:

nowrap - if true then support GZIP compatible compression

Inflater

```
public Inflater()
```

Creates a new decompressor.

Method Detail

setInput

```
public void setInput(byte[] b,
                     int off,
                     int len)
```

Sets input data for decompression. Should be called whenever needsInput() returns true indicating that more input data is required.

Parameters:

b - the input data bytes

off - the start offset of the input data

len - the length of the input data

See Also:

needsInput()

setInput

```
public void setInput(byte[] b)
```

Sets input data for decompression. Should be called whenever needsInput() returns true indicating that more input data is required.

Parameters:

b - the input data bytes

See Also:

needsInput()

setDictionary

```
public void setDictionary(byte[] b,
                          int off,
                          int len)
```

Sets the preset dictionary to the given array of bytes. Should be called when inflate() returns 0 and needsDictionary() returns true indicating that a preset dictionary is required. The method getAdler() can be used to get the Adler-32 value of the dictionary needed.

Parameters:

b - the dictionary data bytes

off - the start offset of the data

len - the length of the data

See Also:

needsDictionary(), getAdler()

setDictionary

```
public void setDictionary(byte[] b)
```

Sets the preset dictionary to the given array of bytes. Should be called when inflate() returns 0 and needsDictionary() returns true indicating that a preset dictionary is required. The method getAdler() can be used to get the Adler-32 value of the dictionary needed.

Parameters:

b - the dictionary data bytes

See Also:

needsDictionary(), getAdler()

getRemaining

public int getRemaining()

Returns the total number of bytes remaining in the input buffer. This can be used to find out what bytes still remain in the input buffer after decompression has finished.

Returns:

the total number of bytes remaining in the input buffer

needsInput

public boolean needsInput()

Returns true if no data remains in the input buffer. This can be used to determine if #setInput should be called in order to provide more input.

Returns:

true if no data remains in the input buffer

needsDictionary

public boolean needsDictionary()

Returns true if a preset dictionary is needed for decompression.

Returns:

true if a preset dictionary is needed for decompression

See Also:

setDictionary(byte[], int, int)

finished

public boolean finished()

Returns true if the end of the compressed data stream has been reached.

Returns:

true if the end of the compressed data stream has been reached

inflate

```
public int inflate(byte[] b,
                   int off,
                   int len)
            throws DataFormatException
```

Uncompresses bytes into specified buffer. Returns actual number of bytes uncompressed. A return value of 0 indicates that needsInput() or needsDictionary() should be called in order to determine if more input data or a preset dictionary is required. In the latter case, getAdler() can be used to get the Adler-32 value of the dictionary required.

Parameters:

`b - the buffer for the uncompressed data`

`off - the start offset of the data`

`len - the maximum number of uncompressed bytes`

Returns:

`the actual number of uncompressed bytes`

Throws:

`DataFormatException - if the compressed data format is invalid`

See Also:

`needsInput()`, `needsDictionary()`

inflate

```
public int inflate(byte[] b)
            throws DataFormatException
```

Uncompresses bytes into specified buffer. Returns actual number of bytes uncompressed. A return value of 0 indicates that needsInput() or needsDictionary() should be called in order to determine if more input data or a preset dictionary is required. In the latter case, getAdler() can be used to get the Adler-32 value of the dictionary required.

Parameters:

`b - the buffer for the uncompressed data`

Returns:

`the actual number of uncompressed bytes`

Throws:

`DataFormatException - if the compressed data format is invalid`

See Also:

`needsInput()`, `needsDictionary()`

getAdler

```
public int getAdler()
```

Returns the ADLER-32 value of the uncompressed data.

Returns:

`the ADLER-32 value of the uncompressed data`

getTotalIn

```
public int getTotalIn()
```

Returns the total number of compressed bytes input so far.

Since the number of bytes may be greater than Integer.MAX_VALUE, the `getBytesRead()` method is now the preferred means of obtaining this information.

Returns:

`the total number of compressed bytes input so far`

getBytesRead

```
public long getBytesRead()
```

Returns the total number of compressed bytes input so far.

Returns:

the total (non-negative) number of compressed bytes input so far

Since:

1.5

getTotalOut

```
public int getTotalOut()
```

Returns the total number of uncompressed bytes output so far.

Since the number of bytes may be greater than Integer.MAX_VALUE, the `getBytesWritten()` method is now the preferred means of obtaining this information.

Returns:

the total number of uncompressed bytes output so far

getBytesWritten

```
public long getBytesWritten()
```

Returns the total number of uncompressed bytes output so far.

Returns:

the total (non-negative) number of uncompressed bytes output so far

Since:

1.5

reset

```
public void reset()
```

Resets inflater so that a new set of input data can be processed.

end

```
public void end()
```

Closes the decompressor and discards any unprocessed input. This method should be called when the decompressor is no longer being used, but will also be called automatically by the finalize() method. Once this method is called, the behavior of the Inflater object is undefined.

finalize

```
protected void finalize()
```

Closes the decompressor when garbage is collected.

Overrides:

`finalize` in class `Object`

See Also:

`WeakReference`, `PhantomReference`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.